PROYECTO BIBLIOTECA IUB

Documentación técnica del proyecto

Estudiante: Isaacs Ferrer
Programa: Ingeniería Telemática
Institución: Institución Universitaria de Barranquilla (IUB)
Asignatura: [Nombre de la asignatura]
Docente: [Nombre del docente]
Fecha de elaboración: 10 de octubre de 2026

Tabla de contenido

Tabla de contenido 2
Introducción 3
Planteamiento del problema 3
Justificación 3
Objetivo general 3
Objetivos específicos 3
Alcance del proyecto 4
Población beneficiaria 4
Requisitos funcionales 4
Requisitos no funcionales 5
Descripción de los módulos existentes 6
Arquitectura del sistema 7
Tecnologías utilizadas 7
Modelo de datos y descripción de las entidades verificadas 8
Autenticación, roles y autorización 8
Descripción del modo de demostración 9
Pruebas realizadas y resultados comprobados 9
Casos de prueba del login y de los permisos 10
Criterios de aceptación 10
Pendientes identificados antes de la presentación 10
Conclusiones 11
Referencias técnicas de las tecnologías utilizadas 11

Introducción

El presente documento corresponde al proyecto “Biblioteca IUB”, un sistema de gestión de acceso y préstamos de recursos informáticos y bibliográficos de la Institución Universitaria de Barranquilla (IUB). El sistema permite autenticación por roles (JWT + RBAC), módulos administrativos y de usuario, gestión de libros, dispositivos y préstamos, servicios de inteligencia artificial para búsqueda y recomendaciones, control de accesos por código QR (kiosco) y un modo demostración separado de la autenticación real. El desarrollo combina un backend FastAPI con PostgreSQL y un frontend Angular, desplegados de forma que el backend corre en Render y la base de datos en Neon.

Planteamiento del problema

La biblioteca de la IUB atiende a estudiantes, docentes y funcionarios en tres sedes (Soledad, Barranquilla y Colsamiro). El control de préstamos de libros y de dispositivos (computadores y tabletas) y el registro de ingresos constituyen procesos que requieren trazabilidad y control de acceso. La ausencia de una plataforma única dificulta: (1) conocer en tiempo real la disponibilidad de ejemplares y equipos, (2) validar la identidad del usuario que solicita un recurso, (3) aplicar reglas de préstamo (plazos 4 a 6 días para libros, franja de máximo 2 horas para dispositivos), y (4) limitar cada usuario a las funciones que le corresponden según su rol. Este proyecto aborda esas necesidades con un sistema web con autenticación por roles y respaldo de datos centralizado.

Justificación

Un sistema centralizado permite agilizar la gestión bibliotecaria, reducir errores manuales y registrar evidencia de cada préstamo y acceso. Al incorporar autenticación con JWT y un modelo de permisos por rol (RBAC), se garantiza que cada usuario solo ejecute las operaciones autorizadas: los estudiantes solicitan recursos, mientras que los administradores gestionan el inventario y aprueban solicitudes. Además, los servicios de búsqueda y recomendación basados en inteligencia artificial mejoran la experiencia del usuario, y el modo demostración permite presentar el sistema sin depender de credenciales reales, manteniendo separados los datos ficticios de los datos reales.

Objetivo general

Desarrollar un sistema integral para la gestión de acceso y préstamos de recursos bibliográficos e informáticos de la Biblioteca IUB, con autenticación basada en roles, módulos administrativos y de usuario, servicios de inteligencia artificial y un modo demostración independiente, respaldado por una base de datos PostgreSQL.

Objetivos específicos

1. Implementar un backend (FastAPI + PostgreSQL) con autenticación JWT, hashing de contraseñas con bcrypt y un modelo de permisos por rol (RBAC) que revalide el rol del usuario desde la base de datos en cada petición protegida.

2. Construir un frontend (Angular) con páginas por rol, guards de navegación, formularios validados y servicios centralizados sobre una sola configuración de URL (environment).

3. Restringir el ingreso al sistema de modo que el rol seleccionado en el login deba coincidir con el rol real de la cuenta.

4. Incorporar un modo demostración separado de la autenticación real, sin persistencia ni llamadas a la API.

5. Integrar servicios de IA (búsqueda semántica, recomendación y estadísticas predictivas) y el registro de accesos por código QR.

6. Desplegar el backend en Render con base de datos en Neon y configurar el pool de conexiones para un servicio gestionado.

Alcance del proyecto

DENTRO del alcance: gestión de usuarios, libros, dispositivos y préstamos; aprobación y devolución de solicitudes; catálogo con búsqueda tradicional y semántica; recomendaciones; panel estadístico; registro de ingresos por DNI/QR; exportación de reportes; autenticación JWT con roles; modo demostración; despliegue del backend en Render y base de datos en Neon.

FUERA del alcance: el consumo desde el frontend de la API Express/MySQL (proveedores y compras) que, aunque está construida, aún no está integrada en la interfaz; el despliegue del frontend en un hosting; la integración con sistemas externos.

Población beneficiaria

Estudiantes, docentes y funcionarios de la Institución Universitaria de Barranquilla que usan los servicios de la biblioteca en las sedes de Soledad, Barranquilla y Colsamiro. El super administrador y los administradores gestionan el sistema; los funcionarios, docentes y estudiantes acceden según sus permisos.

Requisitos funcionales

ID	Requisito funcional	Estado verificado
RF-01	El sistema permite iniciar sesión con identificación y contraseña; la contraseña se valida con bcrypt.	Completo (pytest)
RF-02	El login devuelve un token JWT (HS256, vigencia 8 horas) con el rol del usuario tomado de la base de datos.	Completo (pytest)
RF-03	El rol seleccionado en el formulario debe coincidir con el rol real de la cuenta; si no coincide, se bloquea el ingreso.	Completo
RF-04	El formulario de login incluye un selector de roles y una opción de modo demostración.	Completo

ID	Requisito funcional	Estado verificado
RF-05	El cierre de sesión borra el token, los datos de sesión y desactiva el modo demostración.	Completo
RF-06	Gestionar usuarios (crear, listar, actualizar, eliminar) y limitar la creación de administradores al super administrador.	Completo (pytest)
RF-07	Gestionar libros (listar por sede, crear, actualizar, eliminar, buscar).	Completo
RF-08	Gestionar dispositivos (computadores y tabletas).	Completo
RF-09	Registrar y gestionar préstamos de libros (4 a 6 días) y de dispositivos (máx. 2 horas, horario 8:00–20:00).	Completo (pytest)
RF-10	Aprobar, rechazar, devolver y marcar daño de préstamos por el administrador.	Completo
RF-11	Permitir a un usuario ver únicamente sus propios préstamos; el administrador ve todos.	Completo (pytest)
RF-12	Mostrar catálogo y búsqueda con categorías y búsqueda semántica (IA).	Completo
RF-13	Generar recomendaciones y panel estadístico (tendencia, morosidad, disponibilidad) con IA.	Completo
RF-14	Registrar accesos a la biblioteca por DNI (kiosco QR) y visualizar ingresos con totales.	Completo
RF-15	Exportar reportes de libros, usuarios, dispositivos, préstamos e ingresos (PDF y Excel).	Completo
RF-16	Modo demostración: navega por las pantallas con datos de ejemplo sin llamar a la API ni usar JWT.	Completo
RF-17	Validar y proteger las rutas por rol (guards) en el frontend y por RBAC en el backend.	Completo

Estado “Completo (pytest)” indica que la funcionalidad cuenta con pruebas automatizadas del backend que pasan; “Completo” indica que está implementada y revisada en código. Ninguna funcionalidad ficticia fue incluida.

Requisitos no funcionales

ID	Requisito no funcional
RNF-01	Seguridad: contraseñas con bcrypt; JWT firmado; rol revalidado desde la base de datos en cada petición protegida.

ID	Requisito no funcional
RNF-02	Seguridad: CORS restringido a orígenes configurados; nunca con comodín cuando usa credenciales.
RNF-03	Seguridad: limitador de peticiones por IP para el login y el registro de accesos QR.
RNF-04	Seguridad: los mensajes de error no exponen contraseñas, tokens, IPs ni detalles técnicos.
RNF-05	Rendimiento: pool de conexiones SQLAlchemy con pool_pre_ping y reciclado (5 min) para el servicio gestionado (Neon/PgBouncer).
RNF-06	Disponibilidad: backend desplegado en Render con health check; base de datos en Neon.
RNF-07	Usabilidad: interfaz responsive basada en Bootstrap 5 y tablas con paginación/búsqueda (DataTables).
RNF-08	Compatibilidad: Python 3.11 (Render), PostgreSQL, navegadores modernos.
RNF-09	Mantenibilidad: backend modular (modelos, routers, servicios), nombres descriptivos y pruebas automatizadas.

Descripción de los módulos existentes

El sistema se organiza en módulos que se corresponden con la matriz de permisos (RBAC) del backend:

Módulo	Descripción	Función verificado
login	Autenticación con JWT, selector de rol y modo demo.	Login, roles, demo
inicio	Portal del usuario con recomendaciones.	Recomendaciones IA
catalogo	Listado de libros, categorías y búsqueda semántica.	Catálogo + IA
mis_prestamos	Préstamos de libros del usuario (solicitar, ver).	Préstamos libros
dispositivos	Solicitud de dispositivos con franja y horario.	Préstamos dispositivos
chatbot	Asistente virtual del layout.	Chatbot IA
dashboard	Panel administrativo con indicadores.	Dashboard IA
gestion_usuarios	CRUD de usuarios con jerarquía de roles.	Usuarios
gestion_libros	CRUD de libros y búsqueda.	Libros
gestion_dispositivos	CRUD de dispositivos y gestión de préstamos.	Dispositivos
gestion_prestamos	Aprobar/rechazar/devolver préstamos.	Préstamos
estadisticas	KPIs de accesos y dispositivos.	Estadísticas

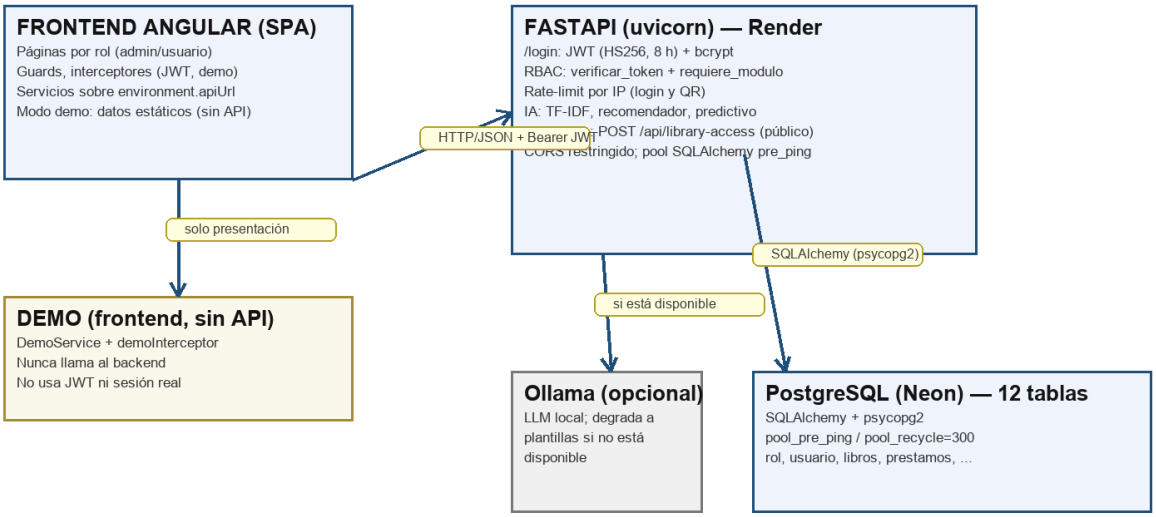
Módulo	Descripción	Función verificado
ingresos	Registro de accesos y lector QR.	Accesos

Arquitectura del sistema

La aplicación sigue una arquitectura de tres capas: un frontend Angular (SPA) que consume una API REST de FastAPI mediante HTTP/JSON e incluye un token JWT en el encabezado Authorization; una API FastAPI desplegada en Render que aplica autenticación, autorización por módulo (RBAC) y reglas de negocio; y una base de datos PostgreSQL en Neon accedida a través de SQLAlchemy con psycopg2 y un pool configurado con pool_pre_ping. Los servicios de inteligencia artificial (búsqueda semántica con TF-IDF, recomendador y predicción) funcionan en el propio backend, con un cliente LLM opcional (Ollama) que degrada automáticamente a plantillas si no está disponible. El modo demostración es exclusivo del frontend e intercepta las peticiones sin contactar el backend.

El siguiente diagrama representa esta arquitectura (elementos comprobados en el código):

Arquitectura de Biblioteca IUB



Tecnologías utilizadas

Tecnología	Uso en el proyecto	Referencia oficial
Angular + TypeScript	Frontend (SPA, componentes standalone, señales, guards, interceptores).	angular.dev
Bootstrap 5 / Bootstrap Icons	Diseño responsive del frontend.	getbootstrap.com
DataTables (jQuery)	Tablas con búsqueda y paginación.	datatables.net
jsPDF / SheetJS	Exportación de reportes PDF y Excel (client-side).	github.com/parallax/jsPDF, sheetjs.com

Tecnología	Uso en el proyecto	Referencia oficial
FastAPI + Uvicorn	API REST del backend.	fastapi.tiangolo.com
SQLAlchemy	ORM y pool de conexiones.	sqlalchemy.org
psycopg2	Controlador PostgreSQL.	psycopg.org
python-jose	Tokens JWT.	github.com/mpdavis/python-jose
passlib + bcrypt	Hashing de contraseñas.	passlib.readthedocs.io / pypi.org/project/bcrypt
scikit-learn / pandas	Búsqueda semántica, recomendación y predicción.	scikit-learn.org / pandas.pydata.org
PostgreSQL	Base de datos.	postgresql.org
Neon	Base de datos gestionada (hosting PostgreSQL).	neon.tech
Render	Hosting del backend.	render.com
Express + MySQL (API auxiliar)	API de proveedores/compras (no integrada aún al frontend).	expressjs.com / mysql.com

Las referencias oficiales indicadas son las fuentes técnicas de cada tecnología; la bibliografía formal debe completarse según el formato exigido por la institución.

Modelo de datos y descripción de las entidades verificadas

El esquema PostgreSQL declarado en el código (12 tablas) coincide con el archivo de esquema “biblioteca_iub_postgres.sql”:

Entidad	Propósito
rol	Roles del sistema (super_administrador, Administrador, Funcionario, Docente, Estudiante).
usuario	Usuarios con identificación, contraseña (hash) y rol por clave foránea.
modulos	Catálogo de módulos de la aplicación.
modulos_X_rol	Matriz de permisos entre roles y módulos (RBAC).
libros	Catálogo de libros con autor, título, programa, ejemplares y sede.
prestamos	Préstamos de libros con estado y solicitud.
dispositivos	Inventario de computadores y tabletas por sede y estado.
prestamos_dispositivos	Solicitudes y gestión de préstamos de dispositivos.
library_access	Registro de accesos a la biblioteca (DNI, mensaje, estado).
contratistas	Proveedores (usado por la API Express auxiliar).
proveer	Relación libros y contratistas.
configuracion	Parámetros de configuración (moneda, horas, plazos).

Autenticación, roles y autorización

La autenticación se realiza en POST /login con identificación y contraseña. La contraseña se verifica con bcrypt; en caso correcto se emite un JWT (HS256) con vigencia de 8 horas y el rol del usuario, que se obtiene de la base de datos. En las peticiones protegidas, la dependencia verificar_token valida la firma y la expiración, y además relee el rol actual del usuario desde la base de datos (si un administrador es degradado, su token deja de actuar como administrador). La autorización se aplica por módulo mediante require_modulo, que consulta la matriz modulos_X_rol. Las operaciones de administración (crear/actualizar/eliminar usuarios, libros, dispositivos y aprobar préstamos) solo están disponibles para roles con el módulo correspondiente; un estudiante obtiene 403 en esas rutas aunque manipule el frontend.

El frontend replica la matriz solo para la navegación (guards y menús). El backend mantiene la autoridad real. Además, el login valida que el rol seleccionado coincida con el rol real devuelto por el backend: si no coinciden, se cierra la sesión y no se navega, mostrando un mensaje de error.

Descripción del modo de demostración

El modo demostración permite recorrer las pantallas con datos de ejemplo sin credenciales y sin contactar la API. Se activa desde el login con un selector de rol y no se restaura al recargar la página (es solo memoria). Sus datos (usuarios, libros, préstamos, accesos, estadísticas) están separados de los datos reales y se sirven mediante un interceptor que responde peticiones falsas; nunca se emite una llamada real al backend. Este modo no cuenta como sesión autenticada: la autenticación real exige un JWT válido.

Pruebas realizadas y resultados comprobados

Las siguientes comprobaciones se ejecutaron durante el desarrollo y se registran como evidencia:

Prueba	Resultado real
Suite de pruebas del backend (pytest, sobre SQLite aislado)	100 pruebas aprobadas
Pruebas de autenticación y autorización (login, JWT, revalidación de rol, 403 por rol)	Aprobadas
Pruebas de préstamos de libros y dispositivos (inventario, franja, privacidad)	Aprobadas
Pruebas de IA (búsqueda semántica, recomendador, predictivo, endpoints /api/ia/*)	Aprobadas
Pruebas de RBAC super administrador y jerarquía de roles	Aprobadas
Pruebas del esquema PostgreSQL base (login contra DDL sin columnas extra)	Aprobadas
Pruebas de configuración del pool y cierre de sesiones SQLAlchemy	Aprobadas
Compilación de producción de Angular (ng build --configuration production)	Exitosa
Verificación de formato de cambios (git diff --check)	Sin errores

Prueba	Resultado real
Pruebas unitarias del frontend (ng test)	No ejecutables: el proyecto no tiene configurado el target test ni archivos .spec

Nota importante: estas pruebas se ejecutan contra una base de datos SQLite aislada; la validación en vivo contra el backend desplegado en Render y la base de datos Neon queda como verificación pendiente de la presentación.

Casos de prueba del login y de los permisos

ID	Caso	Entrada	Resultado esperado	Verificación
CP-01	Login válido con rol coincidente	Credenciales válidas de un Estudiante y rol Estudiante	HTTP 200 + token JWT	Comprobado (pytest)
CP-02	Rol seleccionado distinto del rol real	Credenciales de Estudiante seleccionando Administrador	Bloqueo del ingreso, sesión limpia, sin navegación	Comprobado (implementación); test frontend pendiente
CP-03	Contraseña incorrecta	Identificación válida, contraseña errónea	HTTP 401	Comprobado (pytest)
CP-04	Usuario inexistente	Identificación inexistente	HTTP 401	Comprobado (pytest)
CP-05	Rol enviado por el cliente en el body	Agregar campo rol=super_administrador	Ignorado; responde rol real de la BD	Comprobado (pytest)
CP-06	Estudiante en operaciones administrativas	Token de estudiante en /usuarios/ o PATCH /loan/	HTTP 403	Comprobado (pytest)
CP-07	Rol degradado en BD	Admin es degradado; se reutiliza el token	HTTP 403 (rol revalidado desde BD)	Comprobado (pytest)
CP-08	Token inválido o expirado	Token malformado en /usuarios/	HTTP 401	Comprobado (pytest)
CP-09	Modo demostración	Marcar demo y elegir rol	Navega con datos de ejemplo sin API; no persiste al recargar	Comprobado (implementación); test frontend pendiente

Criterios de aceptación

El sistema se considerará aceptado si: (1) el login valida credenciales reales y bloquea el acceso cuando el rol seleccionado no coincide con el rol de la cuenta; (2) las operaciones administrativas están protegidas por rol en el backend (403 para roles sin permiso); (3) los préstamos respetan las reglas (plazos y franja); (4) el modo demostración funciona sin datos reales ni llamadas a la API; (5) la suite de pruebas del backend pasa al momento de la presentación; y (6) la compilación de producción del frontend finaliza sin errores.

Pendientes identificados antes de la presentación

#	Pendiente	Impacto
1	Verificar en vivo el login y las rutas protegidas contra el backend de Render y Neon.	Confirmación de despliegue
2	Instalar pruebas unitarias del frontend (configurar ng test y archivos .spec).	Evidencia de frontend
3	Añadir pruebas backend para CRUD de libros y dispositivos y estados Completado/Dañado.	Cobertura
4	Decidir el alcance de la API Express/MySQL (Requisito 2) y su consumo desde el frontend.	Requisito 2
5	Completar documentación (README raíz) y la rúbrica del docente si aplica.	Académico
6	Limpiar código duplicado/muerto (badges y funciones sin uso) y revisar la matriz del rol Funcionario.	Calidad
7	Revisar seguridad: enumeración de DNI en el registro público de accesos y secreto JWT por defecto.	Seguridad
8	Commitear el cambio pendiente en el mensaje del login (working tree) y publicar el frente en su hosting.	Entrega final

Conclusiones

El proyecto Biblioteca IUB consolida un sistema funcional de gestión bibliotecaria con autenticación por roles, RBAC revisado desde la base de datos en cada petición, módulos administrativos y de usuario, servicios de inteligencia artificial y un modo demostración separado. La revisión técnica confirmó 100 pruebas aprobadas en el backend, compilación de producción del frontend exitosa y una arquitectura desplegable con el backend en Render y PostgreSQL en Neon. Aunque la funcionalidad central está completa, quedan pendientes verificaciones en vivo, pruebas del frontend y la documentación académica formal; por ello no se afirma un cumplimiento del 100%.

Referencias técnicas de las tecnologías utilizadas

Angular. Documentación oficial. https://angular.dev

Bootstrap. Componentes CSS/JS. https://getbootstrap.com

FastAPI. Framework web. https://fastapi.tiangolo.com

SQLAlchemy. ORM Python. https://www.sqlalchemy.org

psycopg2. Adaptador PostgreSQL. https://www.psycopg.org

PostgreSQL. Base de datos. https://www.postgresql.org

Neon. Postgres gestionado. https://neon.tech

Render. Plataforma de despliegue. https://render.com

scikit-learn. Aprendizaje automático. https://scikit-learn.org

DataTables. Tablas dinámicas. https://datatables.net

Autoría del documento: Issacs Ferrer – Ingeniería Telemática – Institución Universitaria de Barranquilla (IUB).